Knowledge transfer: Nimbus CRM, backend → frontend

From	CRM Platform team (backend)
To	Frontend team, Deals UI project
Product owner	Priya Raghavan, Head of Inside Sales
Status	Backend v1.0 released to the shared dev environment. The frontend starts now
Read with	Stakeholder brief · Backlog · Designs

---

1. Why you are getting this

Inside Sales tracks its deals in a shared spreadsheet. In March a bad sort lost most of a quarter's forecast. Our team has built the backend for deals inside the CRM: the data, the rules, the security and the API. Nothing displays it yet.

Your team owns the user interface from here: React or Angular, your choice, in your own repository. This document is the handover. It tells you:

- what exists
- what the client asked for
- what we did not build
- how we expect to work with you

It does not tell you how to build the screens. That is your design.

2. People

Who	Role	Talk to them about
Priya Raghavan	Product owner, Head of Inside Sales (ADMIN user)	Priorities, acceptance, anything the brief leaves unclear. She is in back-to-backs: batch your questions
Marcus Bell	Sales operations	How the pipeline stages work in practice
Anita Doshi	Sales rep	She reported CRM-142. A good person to test a contacts screen with
Aisha Khan, Ben Carter	Sales reps (USER)	Your day-to-day users
CRM Platform team	Owns the backend	API behaviour, defects, contract questions

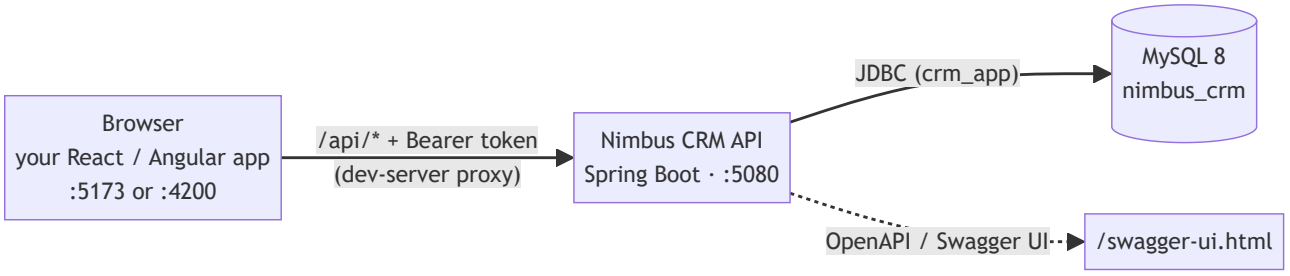
> In the capstone, your mentor plays the product owner and the backend team. Raise questions the way you would at work: in writing, specific, with evidence.

3. Scope: who owns what

Area	Status	Owner
Database schema, seed data, migrations	Done	Backend

Area	Status	Owner
REST API (25 endpoints), validation, business rules	Done	Backend
Authentication (JWT) and authorisation (roles, deal ownership)	Done	Backend
OpenAPI document and Swagger UI	Done	Backend
API test suites (JUnit, end-to-end cross-check script)	Done	Backend
Web UI: every screen, navigation, session handling, error display	**Not started**	**Frontend (you)**
UI tests (unit, component, end-to-end)	**Not started**	**Frontend (you)**
Changes to the API	Not planned this release	Backend, on request with evidence

4. System overview



There is one environment: everyone runs the API locally against their own MySQL. The steps are in the README run book:

1. Create the database (once).
2. Start the API with `mvn spring-boot:run` in `backend/`.
3. Check it: Swagger UI loads, and `node scripts/crosscheck-backend.mjs` reports all checks passing.

Point your dev server's proxy at `http://localhost:5080` for `/api`. If you call the API directly instead, CORS allows `localhost:5173`, `5176` and `4200`.

5. Authentication and authorisation

- `POST /api/auth/login` with `{email, password}` returns `{token, expiresAt, user: {userRef, displayName, email, role}}`.
- Send `Authorization: Bearer <token>` on every other call. Tokens are HS256 JWTs and last **8 hours**. There is no refresh token: when a token expires, the API answers **401** and the user signs in again.
- `GET /api/auth/me` returns the current user, so you can restore a session after a page reload.

Capability	ADMIN	USER (sales rep)
Read accounts, contacts, activities	✔	✔
Log an activity	✔	✔
See deals	all	**only deals they own**
Move a deal's stage	any deal	own deals
Create / edit / delete deals, choose owner	✔	✗ 403

Capability	ADMIN	USER (sales rep)
Create / edit accounts and contacts, import CSV	✔	✗ 403
List users (`GET /api/users`)	✔	✗ 403

Ownership is enforced on the server. A rep who asks for another rep's deal gets **404**, the same answer as for a deal that doesn't exist, so the API never confirms that a deal is there. Hiding a button in the UI is good UX. It is not security.

6. API contract

The source of truth is the running OpenAPI document: Swagger UI at `/swagger-ui.html`, and the JSON at `/v3/api-docs`. Every endpoint is also ready to send in `backend/api-tests.http`.

Area	Endpoints
Auth	`POST /auth/login` · `GET /auth/me` · `GET /users` (ADMIN)
Accounts	`GET /accounts` · `GET /accounts/{ref}` · `GET /accounts/{ref}/contacts` · `POST /accounts` (ADMIN) · `PUT /accounts/{ref}` (ADMIN)
Contacts	`GET /contacts?page&size&status` · `GET /contacts/search?q` · `GET /contacts/{ref}` · `POST /contacts` (ADMIN) · `PUT /contacts/{ref}` (ADMIN) · `POST /contacts/import` (ADMIN, `text/csv`)
Activities	`GET /contacts/{ref}/activities` · `POST /contacts/{ref}/activities`
Deals	`GET /deals?page&size&stage` · `GET /deals/summary` · `GET /deals/{ref}` · `GET /deals/{ref}/history` · `GET /contacts/{ref}/deals` · `POST /deals/{ref}/stage` · `POST /deals` (ADMIN) · `PUT /deals/{ref}` (ADMIN) · `DELETE /deals/{ref}` (ADMIN)

All paths are under `/api`. That is **25 endpoints** in total.

Conventions

- **Identifiers.** Records are addressed by business reference: `A-0008`, `C-00089`, `D-00016`, `U-0002`. Database ids never leave the API.
- **Paging.** Paged lists take `page` (0-based) and `size` (1–100, default 20). They return `{items, page, size, totalElements, totalPages}`.
- **Money.** `value` is a JSON number in GBP with two decimal places. `currency` is always `"GBP"`.
- **Dates and times.** Calendar dates are `yyyy-MM-dd`. Timestamps are ISO-8601 in UTC (`…Z`).
- **Enum values** are upper case:
  - stages: `PROSPECTING`, `QUALIFIED`, `PROPOSAL`, `NEGOTIATION`, `WON`, `LOST`
  - contact statuses: `PROSPECT`, `CUSTOMER`, `CHURNED`
  - activity types: `CALL`, `EMAIL`, `MEETING`, `NOTE`
- **Errors** come in two shapes:

```
// 400: one message per field, written to be shown to a user
{ "error": "VALIDATION_FAILED", "fields": { "value": "value must be greater than zero" } }
// 401, 403, 404, 409: a code to branch on, and a message to show
{ "error": "DEAL_STAGE_TRANSITION_INVALID", "message": "Cannot move D-00001 from PROSPECTING to NEGOTIATION. A
```

The error codes are:

- `UNAUTHENTICATED`
- `INVALID_CREDENTIALS`
- `FORBIDDEN`

- `ACCOUNT_NOT_FOUND`, `CONTACT_NOT_FOUND`, `DEAL_NOT_FOUND`
- `DEAL_STAGE_TRANSITION_INVALID`, `DEAL_CLOSED`, `DEAL_DELETE_NOT_ALLOWED`
- `VALIDATION_FAILED`
- `IMPORT_FORMAT_INVALID`

7. Data model

There are **10 tables and 60 columns**:

- six business tables;
- four lookup tables that fix the allowed values;
- 12 foreign keys linking them.

The full data dictionary, with the reasoning behind each key and constraint, is in `backend/docs/DATABASE.md`.

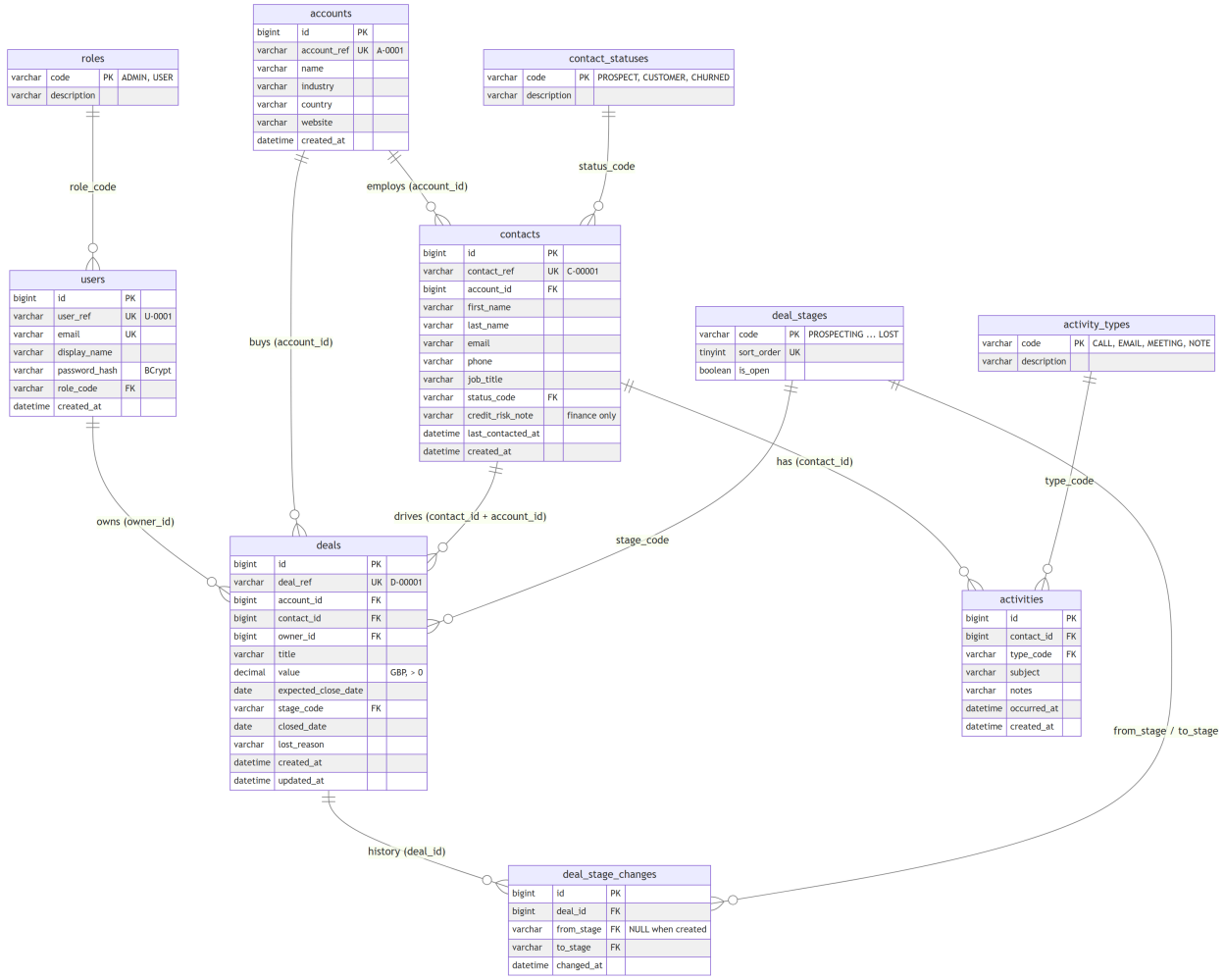


Table	One row is	Notes for the UI
`users`	someone who can sign in	Role ADMIN or USER. Owns deals
`accounts`	a company we sell to	30 seeded. There is no delete

Table	One row is	Notes for the UI
`contacts`	a person at an account	150 seeded. `email` is not unique. **`credit_risk_note` is finance-only: it must not be shown to sales staff or customers**
`activities`	a call, email, meeting or note	Logging one updates the contact's `last_contacted_at`
`deals`	a sale in progress	GBP value > 0. The contact must work at the deal's account. Has one owner
`deal_stage_changes`	one move of one deal	Written by the server. Read-only for you
`roles`, `contact_statuses`, `activity_types`, `deal_stages`	an allowed value	Your dropdowns must offer exactly these values

8. Business rules the server enforces

1. A deal moves **one stage forward or one stage back** at a time. It can go to **LOST** from any open stage, and to **WON** only from **NEGOTIATION**. **WON** and **LOST** are final.
2. Winning a deal needs a `closeDate`. Losing one needs a `lostReason`.
3. Winning a deal makes its contact a **CUSTOMER**. Losing a deal never reverses that.
4. A new deal starts in **PROSPECTING**. Its `expectedCloseDate` can't be in the past, and its contact must work at its account.
5. Only a **PROSPECTING** deal can be deleted. A WON or LOST deal can't be edited.
6. Every stage change is recorded with the from-stage, the to-stage and the time.
7. `GET /deals/summary` gives the count and value per stage and the **open** total (the four open stages). It is scoped to the caller: a rep sees their own figures.

Breaking a rule returns 400 or 409 with a message written for the end user. Show it. Don't rewrite it.

9. Not in this release

The brief asks for these, and the backend **does not support** them. Don't build workarounds without the product owner's agreement:

Asked for	Status
"Who moved it" in the stage history	Not recorded. The history has the time and the stages only
Amounts in each deal's own currency	Not supported. GBP only
A board that updates in real time	Not supported. The API is plain request and response
Deleting accounts or contacts	No endpoint, by design

10. Test data

Sign in as	Password	Role	Sees
`admin@nimbus.test` (Priya)	`Admin#2026`	ADMIN	all 32 deals
`aisha@nimbus.test`	`Rep#2026`	USER	her 16 deals

Sign in as	Password	Role	Sees
`ben@nimbus.test`	`Rep#2026`	USER	his 16 deals

`db-setup/02-reset-schema.mysql.sql`, followed by an API restart, puts the data back to this state. These are local development credentials only.

11. How we work together

- **Backlog:** `docs/backlog/`. Every piece of work is a ticket (`NIM-1xx`) with acceptance criteria. Pick up tickets in sprint order (the sprint goals and rules are in the backlog index).
- **Designs:** `docs/designs/` shows the **design intent** for the main screens. It isn't pixel-perfect. Any state not shown (loading, empty, errors, the rep's view) is for you to design.
- **Specification first:** NIM-100 (the spec, using `docs/specs/_template.md`) is done before any screen ticket starts.
- **Branches and PRs:**
  - Work on one branch per ticket (`feature/NIM-112-contact-profile`).
  - Open one pull request per ticket, with the ticket ID in the title.
  - Show the acceptance criteria and the evidence that each one is met (screenshots, test names).
- **AI assistants (GitHub Copilot) are expected.**
  - You own every line you merge.
  - Keep a short **prompt journal** per ticket: what you asked, what you kept, and what you changed and why.
  - Never paste tokens, passwords or customer data into a prompt.
- **Backend defects:** if the API looks wrong, raise a bug with the request, the response, what you expected and why, and the impact. Use the guide in `backend/docs/API-TESTING.md`. Don't work around it silently, and don't edit the backend.
- **Definition of done** (per ticket):
  - The acceptance criteria are met for each role that can see the feature.
  - The loading, empty and error states exist.
  - Automated tests cover the acceptance criteria.
  - It works with the keyboard alone.
  - No secrets or restricted data appear in the UI, the console or URLs.
  - The PR has been reviewed against `docs/review-checklist.md`.

12. Known open items from the backend team

- **CRM-142**: Inside Sales report that the contact list is out of order. It has been closed twice as "cannot reproduce". Ticket NIM-114 asks you to investigate it from the UI side. The report is in `bug-CRM-142.md`.
- The stage rules in section 8 come from Marcus. The brief doesn't list them. Confirm the user-facing wording with Priya.
- We have tested the API with our scripts, not with a real UI. Expect to find things. When you do, tell us.

Backlog

Every piece of work for the Deals UI project, in three sprints. Pick tickets up in sprint order and respect the dependencies. Each ticket is done when its acceptance criteria are met and the definition of done in the KT handover is satisfied.

Sprint 1: A signed-in rep can find any customer and see the whole relationship.

Ticket	Type	Epic	Priority	Points
NIM-100 · Explore the API and write the specification	Spike	Quality & release	Highest	3
NIM-101 · Front-end project setup	Task	Quality & release	Highest	2
NIM-102 · Sign in	Story	Access & identity	Highest	3
NIM-103 · Session lifetime and sign-out	Story	Access & identity	High	3
NIM-104 · Only show what my role allows	Story	Access & identity	High	3
NIM-110 · Contact list	Story	Customers (contacts & accounts)	High	3
NIM-111 · Find a contact	Story	Customers (contacts & accounts)	High	2
NIM-112 · Contact profile	Story	Customers (contacts & accounts)	High	5
NIM-113 · Accounts and the people at them	Story	Customers (contacts & accounts)	Medium	3
NIM-114 · CRM-142: Contacts list is not in the order it claims to be	Bug	Customers (contacts & accounts)	High	2

Sprint 1 total: 29 points

Sprint 2: Reps run their own pipeline in the CRM, and Priya has her Monday number.

Ticket	Type	Epic	Priority	Points
NIM-120 · Deal list	Story	Deal pipeline	Highest	5
NIM-121 · Deal details and history	Story	Deal pipeline	High	3
NIM-122 · Move a deal through the pipeline	Story	Deal pipeline	Highest	5
NIM-123 · Pipeline board and the Monday number	Story	Deal pipeline	Highest	5
NIM-124 · Deals and activity on the contact page	Story	Deal pipeline	High	3
NIM-125 · Home page	Story	Deal pipeline	Medium	3

Sprint 2 total: 24 points

Sprint 3: Priya manages the data herself, the app is tested, and we release.

Ticket	Type	Epic	Priority	Points
NIM-130 · Create, edit and delete deals	Story	Data management (admin)	Highest	5
NIM-131 · Assign a deal owner	Story	Data management (admin)	High	2
NIM-132 · Add and edit accounts	Story	Data management (admin)	Medium	3
NIM-133 · Add and edit contacts	Story	Data management (admin)	Medium	3
NIM-134 · Import leads from the agency	Story	Data management (admin)	Low	5
NIM-140 · Automated tests, then audit them	Task	Quality & release	High	5
NIM-141 · Review and harden	Task	Quality & release	High	3
NIM-142 · Release	Task	Quality & release	High	2

Sprint 3 total: 28 points

Demos

Sprint	Demo to Priya
1	Sign in as a rep, find Nadia Fenwick, open her profile and her company
2	Aisha works her pipeline; Priya reads her Monday number
3	Priya creates a deal for Ben; Ben finds it; test evidence and release notes

How the sprints run

- **Planning** at the start of each sprint: confirm the goal, pull the tickets, raise questions for Priya.
- **Stand-up** daily: done, next, blocked. Raise blockers the same day.
- **Refinement** mid-sprint: read the next sprint's tickets and send your questions to Priya and the backend team in writing.
- **Review** at the end: demo working software against each ticket's acceptance criteria. Each ticket is accepted or rejected.
- **Retrospective** at the end: one thing to change next sprint.

Rules

- NIM-100 (the specification) is finished before any screen ticket starts.
- A ticket is in progress only when someone owns it and it has a branch.
- A ticket is done only when its acceptance criteria are shown working against the real API.
- A ticket that is not done goes back to the backlog with a note. It is never "done except for tests".

By epic

- **Access & identity:** NIM-102, NIM-103, NIM-104
- **Customers (contacts & accounts):** NIM-110, NIM-111, NIM-112, NIM-113, NIM-114
- **Deal pipeline:** NIM-120, NIM-121, NIM-122, NIM-123, NIM-124, NIM-125
- **Data management (admin):** NIM-130, NIM-131, NIM-132, NIM-133, NIM-134
- **Quality & release:** NIM-100, NIM-101, NIM-140, NIM-141, NIM-142

*Ticket types: **Story**, user-facing value; **Task**, technical work; **Spike**, timeboxed investigation; **Bug**, a reported defect.*

NIM-100 · Explore the API and write the specification

Type	Spike
Epic	Quality & release
Sprint	Sprint 1
Priority	Highest
Story points	3
Depends on	—

Context

Before any screen is built, the team needs a shared understanding of what the client asked for, what the backend supports, and where the two disagree. Priya's brief (`docs/stakeholder-brief.md`) is an email, not a specification.

Acceptance criteria

```
Scenario: Specification exists before screen work starts
  Given the stakeholder brief, the KT handover and the running API
  When the spike is complete
  Then docs/specs/ contains a specification using docs/specs/_template.md
  And every brief requirement is marked supported, partly supported or not supported, with evidence (a request
  And every ambiguity or contradiction in the brief is listed with the question to ask Priya
  And the agreed answers are recorded once Priya has replied
```

Backend notes

Swagger UI `/swagger-ui.html` , `backend/api-tests.http` .

Out of scope

Writing UI code.

Notes

Timebox: 1 day. This is the one process rule that is assessed: no screen ticket starts before this is done.

NIM-101 · Front-end project setup

Type	Task
Epic	Quality & release
Sprint	Sprint 1
Priority	Highest
Story points	2
Depends on	NIM-100

Context

Create the front-end repository the team will work in. React or Angular is the team's choice; record the decision and why in the README.

Acceptance criteria

```
Scenario: A new developer can run the app
  Given a fresh clone of the front-end repository and the API running on port 5080
  When they follow the README
  Then the app starts locally and /api requests reach the API without CORS errors
  And a unit test command and a lint or type-check command both run and pass

Scenario: Agreed structure
  Then all HTTP calls go through one place in the codebase
  And the shapes of API responses are typed
```

Backend notes

Proxy `/api` to `http://localhost:5080`. Direct calls are allowed from ports 5173, 5176 and 4200.

Out of scope

Screens.

NIM-102 · Sign in

Type	Story
Epic	Access & identity
Sprint	Sprint 1
Priority	Highest
Story points	3
Depends on	NIM-101

User story

> As a member of the Inside Sales team, I want to sign in with my work email and password, so that only Nimbus staff can see customer data.

Context

Every API call except sign-in now requires a token. Until the user signs in, the app can show nothing else.

Acceptance criteria

```
Scenario: Successful sign-in
  Given I am signed out
  When I sign in as aisha@nimbus.test with the correct password
  Then I land on the page I originally asked for, or the home page if I asked for none

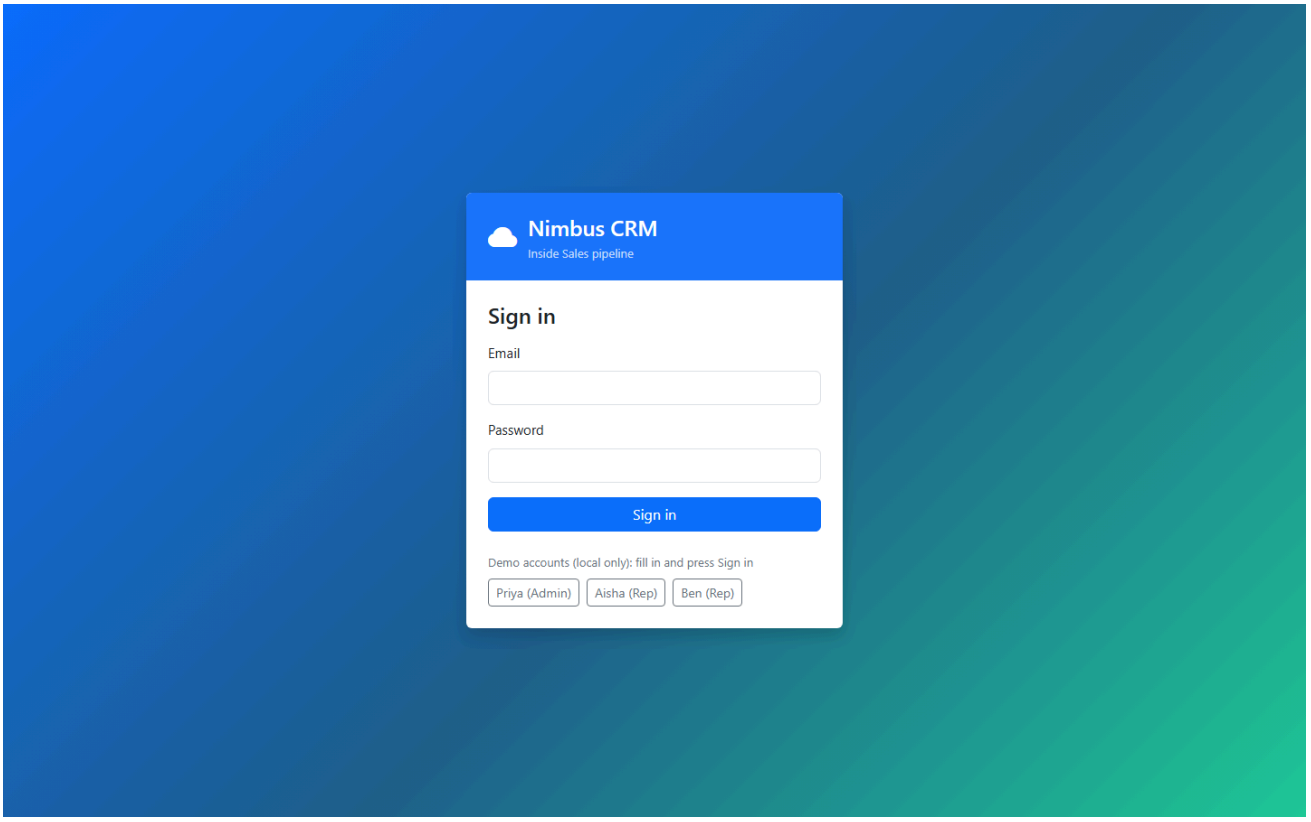
Scenario: Wrong credentials
  When I sign in with a wrong password
  Then I am told the email or password is incorrect, without being told which
  And the email I typed is kept and the password is cleared

Scenario: Missing input
  When I submit with the email or password empty
  Then I am told which field is missing, and no request is sent

Scenario: Unsafe return address
  Given I followed a sign-in link whose return address points to another website
  When I sign in
  Then I land on the home page of this app
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`POST /api/auth/login` → `{token, expiresAt, user}`. Wrong credentials → 401 `INVALID_CREDENTIALS`. Empty body → 400 with `fields.email` / `fields.password`.

Out of scope

Password reset, "remember me", self sign-up.

NIM-103 · Session lifetime and sign-out

Type	Story
Epic	Access & identity
Sprint	Sprint 1
Priority	High
Story points	3
Depends on	NIM-102

User story

> As a signed-in user, I want to stay signed in while I work and to know who I am signed in as, so that I am not interrupted and never act as the wrong person.

Context

Tokens last 8 hours and cannot be refreshed. Reps leave the CRM open all day.

Acceptance criteria

```
Scenario: Reload keeps me signed in
  Given I am signed in
  When I reload the browser tab
  Then I am still signed in as the same user

Scenario: Expired session
  Given my session has expired
  When the app makes any request
  Then I am taken to sign-in with a message that my session has ended
  And after signing in again I return to the page I was on

Scenario: Who am I
  Then my name and role are visible on every page

Scenario: Sign out
  When I sign out
  Then I am on the sign-in page
  And pressing Back or opening a bookmarked page does not show customer data
```

Backend notes

`GET /api/auth/me`. Any call with a missing or expired token → 401 `UNAUTHENTICATED`.

Out of scope

Single sign-on, multi-tab session sync.

NIM-104 · Only show what my role allows

Type	Story
Epic	Access & identity
Sprint	Sprint 1
Priority	High
Story points	3
Depends on	NIM-103

User story

> As a sales rep, I want to be offered only the actions I am allowed to take, so that I am not confused by buttons that fail.

Context

Priya: "Reps should only see their own deals. Right now everyone can see everything and it causes arguments about who owns what." Admins manage data; reps work their own deals. The server enforces this whatever the UI shows.

Acceptance criteria

```
Scenario: Rep does not see admin actions
  Given I am signed in as a rep
  Then I am never offered create, edit or delete for deals, contacts or accounts

Scenario: Rep opens an admin page directly
  Given I am signed in as a rep
  When I open the address of an admin-only page
  Then I am told I do not have access, and no admin form is shown

Scenario: Server refuses an action
  When the API answers 403 to anything I do
  Then I see a clear "not allowed" message and stay where I am
```

Backend notes

See the KT handover, section 5 (role matrix). 403 `FORBIDDEN`.

NIM-110 · Contact list

Type	Story
Epic	Customers (contacts & accounts)
Sprint	Sprint 1
Priority	High
Story points	3
Depends on	NIM-104

User story

> As a rep, I want to browse contacts in surname order, a page at a time, and filter them by status, so that I can work through the people we sell to.

Context

This screen replaces the contacts tab of the old spreadsheet. Inside Sales expect surname order.

Acceptance criteria

```
Scenario: Browse
  Given I am signed in
  When I open the contacts list
  Then I see 20 contacts per page with name, account, job title and status
  And I can move to the next and previous pages and see which page I am on out of how many

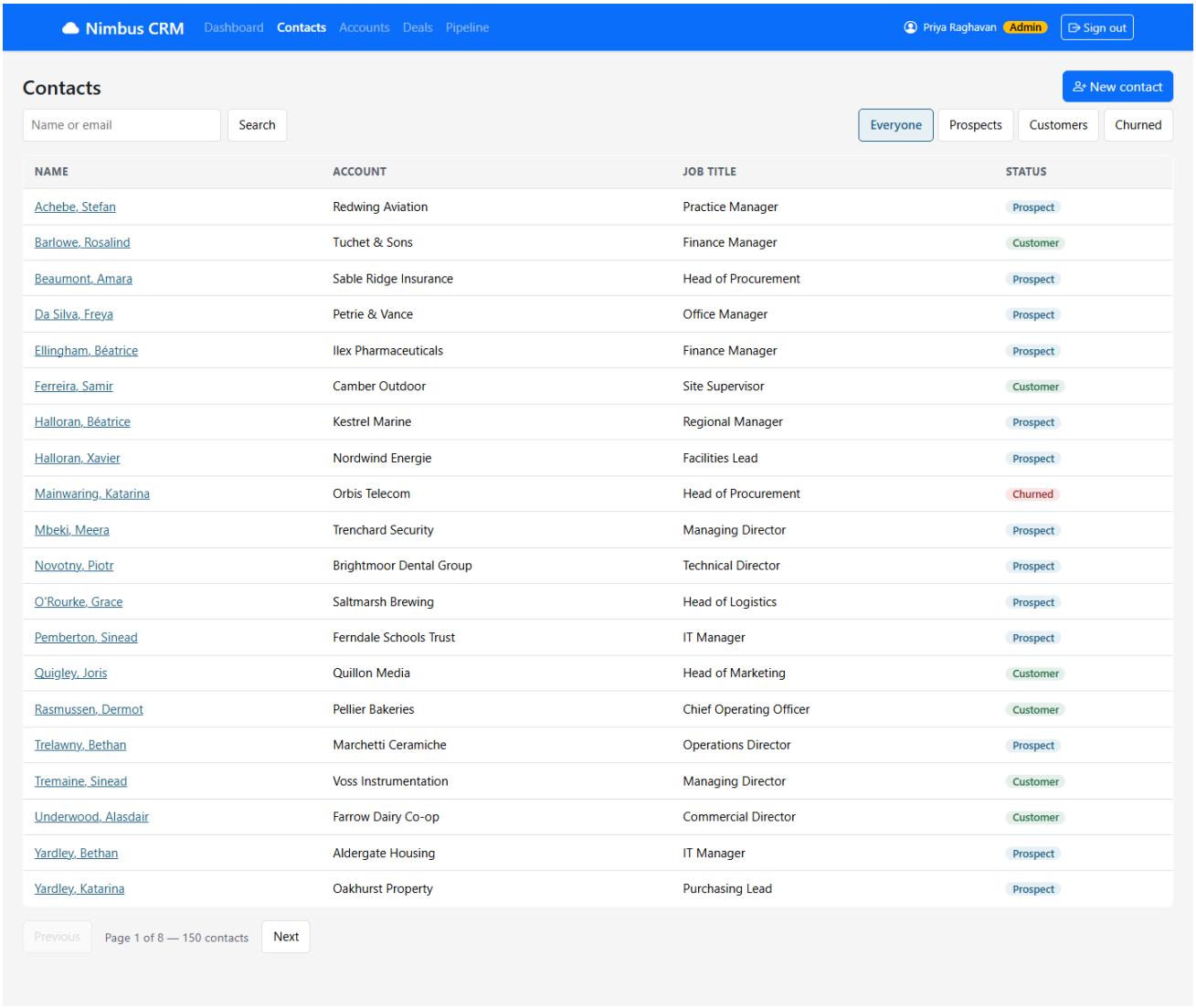
Scenario: Filter by status
  When I filter by Customer
  Then only customers are listed, and paging applies to the filtered list

Scenario: Open a contact
  When I choose a contact
  Then I see their profile (NIM-112)

Scenario: Nothing to show
  When a filter matches nobody
  Then I am told so
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/contacts?page=&size=&status=` (status: `PROSPECT`, `CUSTOMER`, `CHURNED`).

Notes

See NIM-114 before you sign this off.

NIM-111 · Find a contact

Type	Story
Epic	Customers (contacts & accounts)
Sprint	Sprint 1
Priority	High
Story points	2
Depends on	NIM-110

User story

> As a rep, I want to find a contact by name or email, so that I can get to someone quickly while they are on the phone.

Context

Anita: "This morning I went looking for Nadia Fenwick… then gave up and clicked through every page one at a time."

Acceptance criteria

```
Scenario: Search
  When I search for "fenwick"
  Then every contact whose first name, last name or email contains "fenwick" is listed

Scenario: No match
  When I search for something that matches nobody
  Then I am told so

Scenario: Back to browsing
  When I clear the search
  Then I return to the paged list
```

Backend notes

`GET /api/contacts/search?q=` (not paged).

NIM-112 · Contact profile

Type	Story
Epic	Customers (contacts & accounts)
Sprint	Sprint 1
Priority	High
Story points	5
Depends on	NIM-110

User story

> As a rep, I want to open a contact and see their details, their company and everything we have logged with them, so that I know where the relationship stands before I call.

Context

The contact record is the heart of the CRM. Finance also keep information against contacts that is for their eyes only (see the data model in the KT handover).

Acceptance criteria

```
Scenario: Profile
  When I open contact C-00089
  Then I see their name, job title, email, phone, status, when they were last contacted and their company
  And I can go from the contact to their company

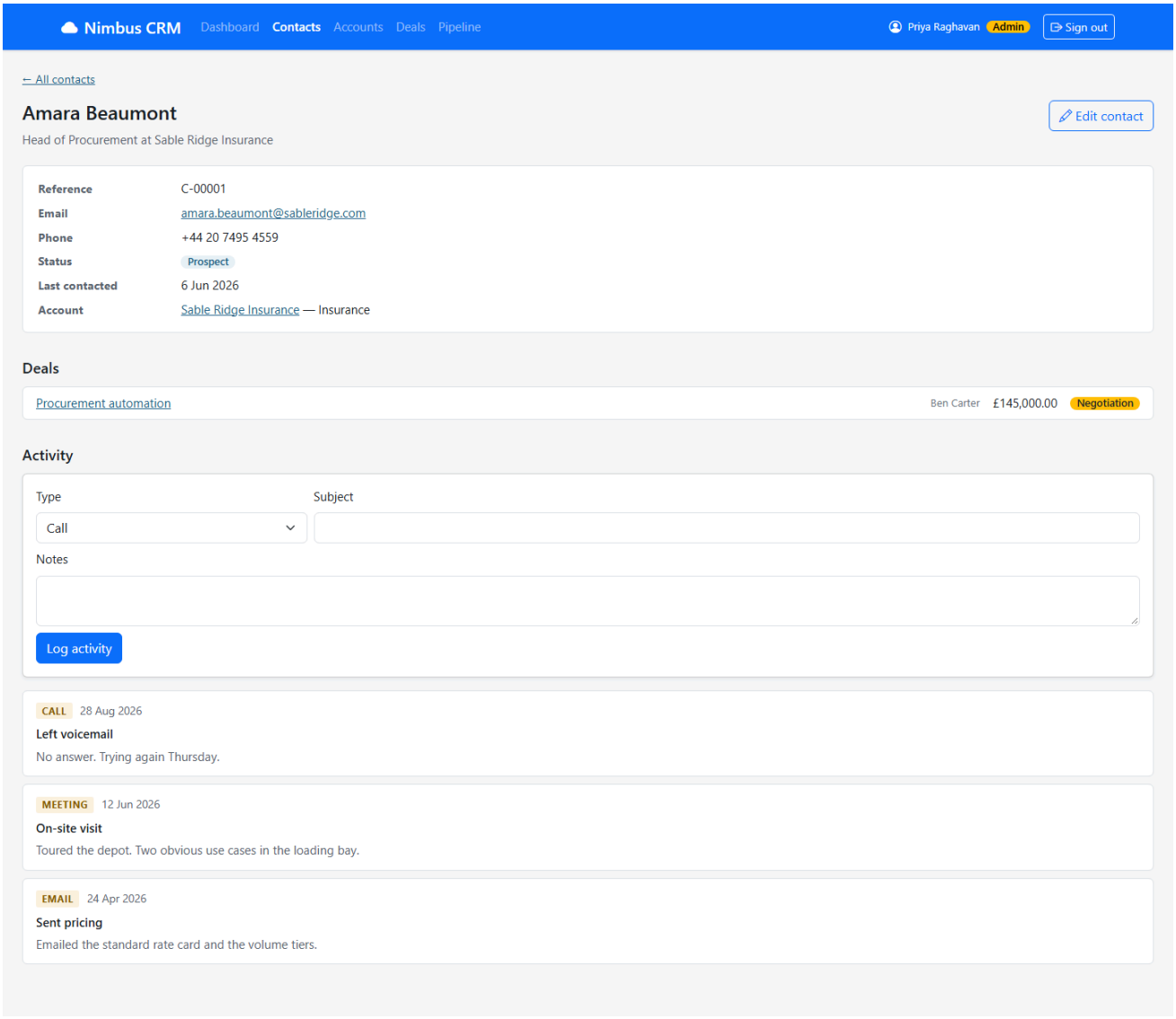
Scenario: Activity history
  Then I see every logged activity, newest first, with type, date, subject and notes
  And if nothing has been logged I am told so

Scenario: Unknown contact
  When I open a contact reference that does not exist
  Then I am told it was not found and can get back to the list

Scenario: Restricted information
  Then nothing classified as finance-only is visible to me, in the page or anywhere else the browser keeps it
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/contacts/{ref}`, `GET /api/contacts/{ref}/activities`. Unknown ref → 404 `CONTACT_NOT_FOUND`.

Out of scope

Editing (NIM-133), deals on this page (NIM-124).

NIM-113 · Accounts and the people at them

Type	Story
Epic	Customers (contacts & accounts)
Sprint	Sprint 1
Priority	Medium
Story points	3
Depends on	NIM-112

User story

> As a rep, I want to see our accounts and the people at each one, so that I understand who I am dealing with at a company.

Context

There are 30 accounts. Reps mostly arrive at an account from a contact.

Acceptance criteria

```
Scenario: Account directory
  When I open accounts
  Then I see every account with industry, country and website

Scenario: Account page
  When I open account A-0008
  Then I see its details and the people who work there, each leading to their profile
```

Backend notes

`GET /api/accounts`, `GET /api/accounts/{ref}`, `GET /api/accounts/{ref}/contacts`.

Out of scope

Creating or editing accounts (NIM-132).

NIM-114 · CRM-142: Contacts list is not in the order it claims to be

Type	Bug
Epic	Customers (contacts & accounts)
Sprint	Sprint 1
Priority	High
Story points	2
Depends on	NIM-110

Context

Reported three times by Inside Sales, closed twice as "cannot reproduce". The original report is in `docs/bug-CRM-142.md`.

Acceptance criteria

```
Scenario: Reproduced or ruled out
  Given the seeded data
  When the defect is investigated
  Then the report states whether it reproduces, with the exact requests and responses as evidence
  And it states whether the cause is in the UI or the API

Scenario: If the cause is in the API
  Then a bug report is raised to the backend team (see backend/docs/API-TESTING.md)
  And the contacts screen does not make the problem worse or hide it from users
  And the team agrees with Priya what users should be offered meanwhile
```

Backend notes

Report API defects; do not change the backend.

Notes

The previous two investigations only looked at the first page.

NIM-120 · Deal list

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	Highest
Story points	5
Depends on	NIM-104

User story

> As a rep, I want to see my deals, soonest closing first, and filter them by stage, so that the spreadsheet is no longer needed.

Context

Priya: "I want to open the CRM and see our deals, the same way I can see our contacts." Reps see their own deals; Priya sees everyone's.

Acceptance criteria

```
Scenario: My deals
  Given I am signed in as aisha@nimbus.test
  When I open deals
  Then I see only deals I own: 16 on a freshly seeded database
  And each shows title, account, contact, value in pounds, expected close date, stage and owner

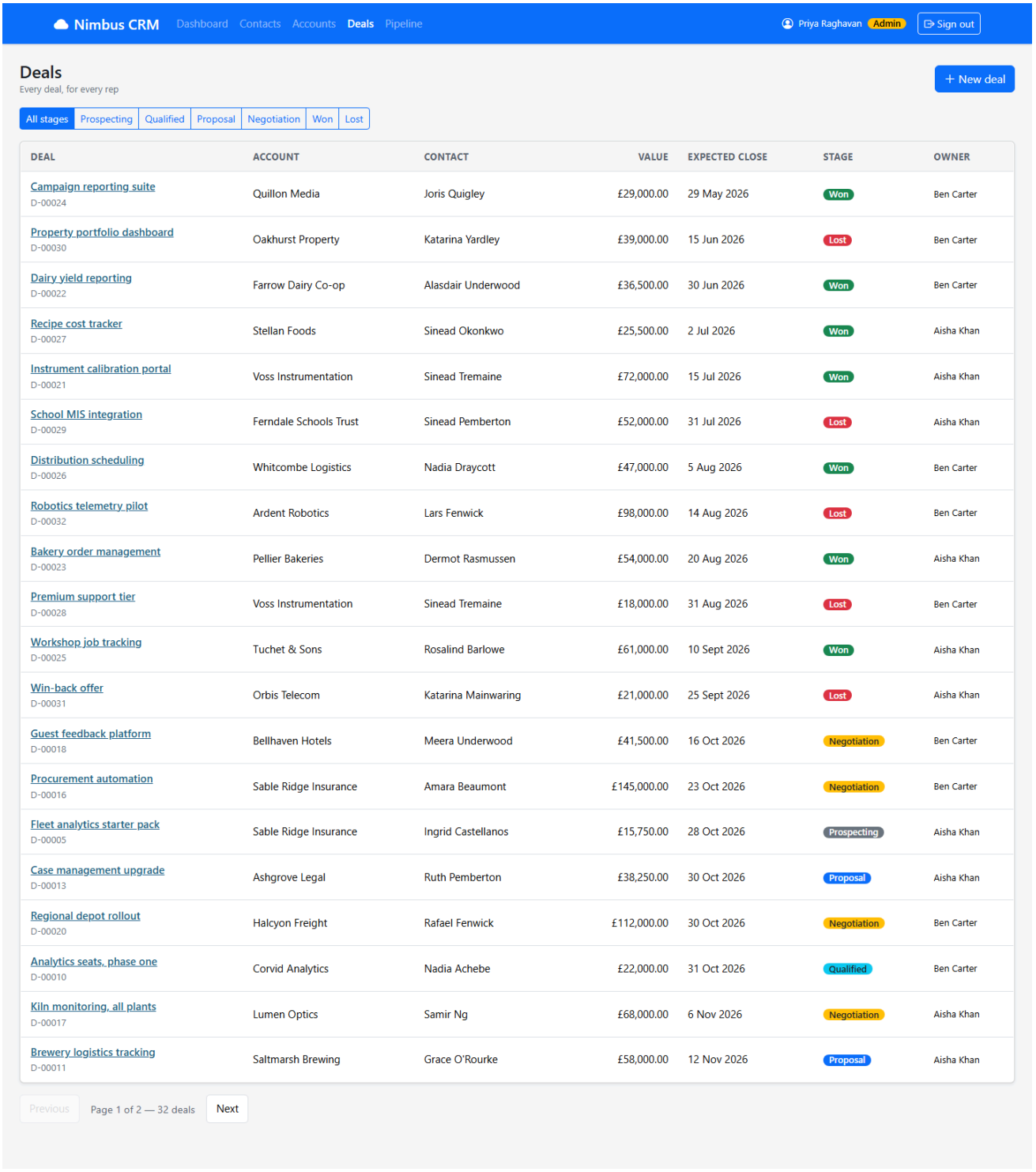
Scenario: Everyone's deals
  Given I am signed in as Priya
  Then I see all 32 deals

Scenario: Filter and page
  When I filter by Negotiation
  Then only deals in Negotiation are listed
  And the filter and page survive a browser reload and can be shared as a link

Scenario: Open a deal
  When I choose a deal
  Then I see its details (NIM-121)
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/deals?page=&size=&stage=`. An unknown stage → 400 with `fields.stage`.

NIM-121 · Deal details and history

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	High
Story points	3
Depends on	NIM-120

User story

> As a rep, I want to see everything about a deal, including how it has moved through the pipeline, so that I can pick up where things stand.

Context

Priya: "I need full audit history of every stage change — who moved it, when, and from what to what. Compliance have asked for this twice."

Acceptance criteria

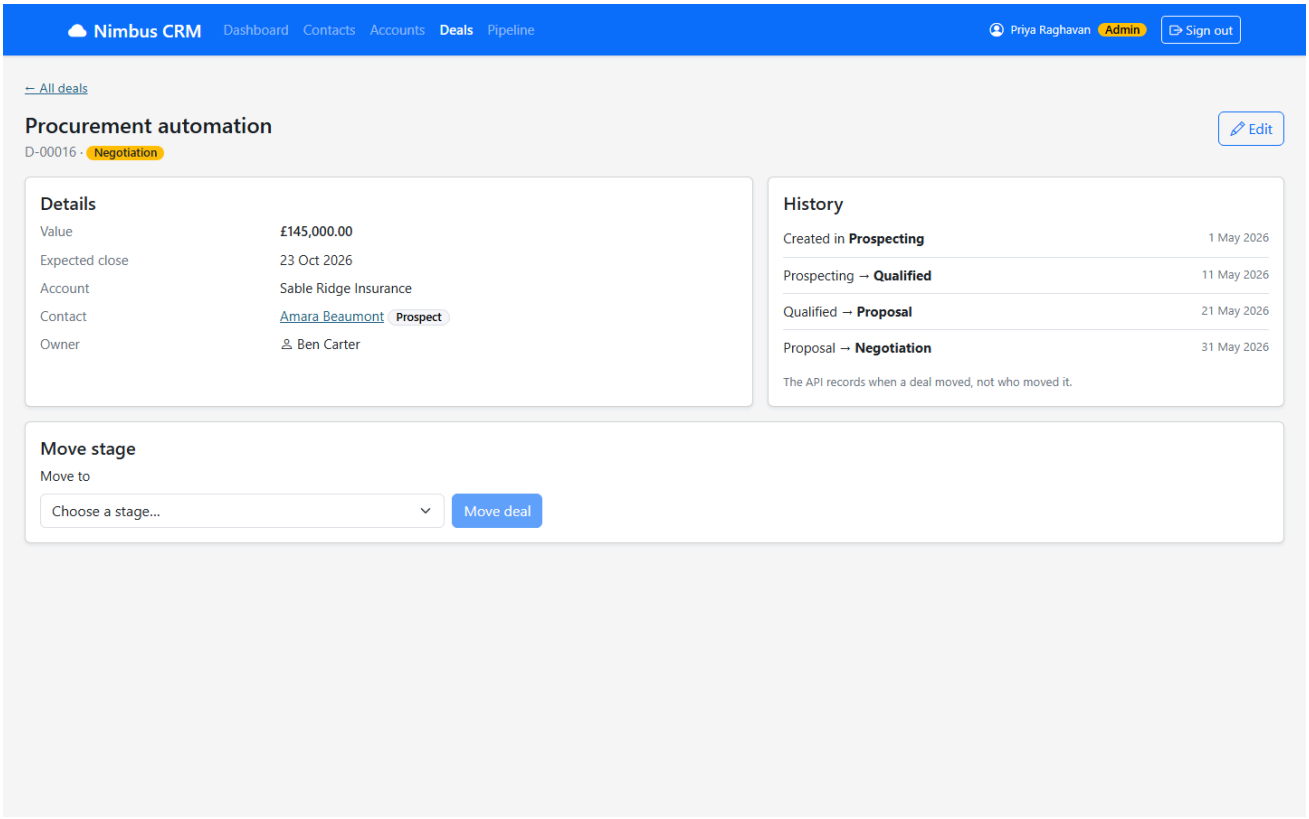
```
Scenario: Details
  When I open deal D-00016 as Priya
  Then I see its value, expected close date, stage, account, contact (with their current status) and owner
  And for a closed deal, the close date and, if lost, the reason

Scenario: History
  Then I see every stage change, oldest first, starting from when it was created

Scenario: Not found or not mine
  Given I am signed in as Aisha
  When I open a deal that does not exist, or one that Ben owns
  Then I am told it was not found
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/deals/{ref}`, `GET /api/deals/{ref}/history`. Not found or not yours → 404 `DEAL_NOT_FOUND`.

NIM-122 · Move a deal through the pipeline

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	Highest
Story points	5
Depends on	NIM-121

User story

> As a rep, I want to move my deal to its next stage, or back, or close it, so that the pipeline is accurate.

Context

Priya: "It should not be possible to put a deal in a silly state. Deals do go backwards sometimes… but obviously not just anything to anything. When a deal is won I want that reflected on the customer record."

Acceptance criteria

```
Scenario: Allowed moves only
  Given a deal in an open stage
  Then I am only offered the moves the business allows

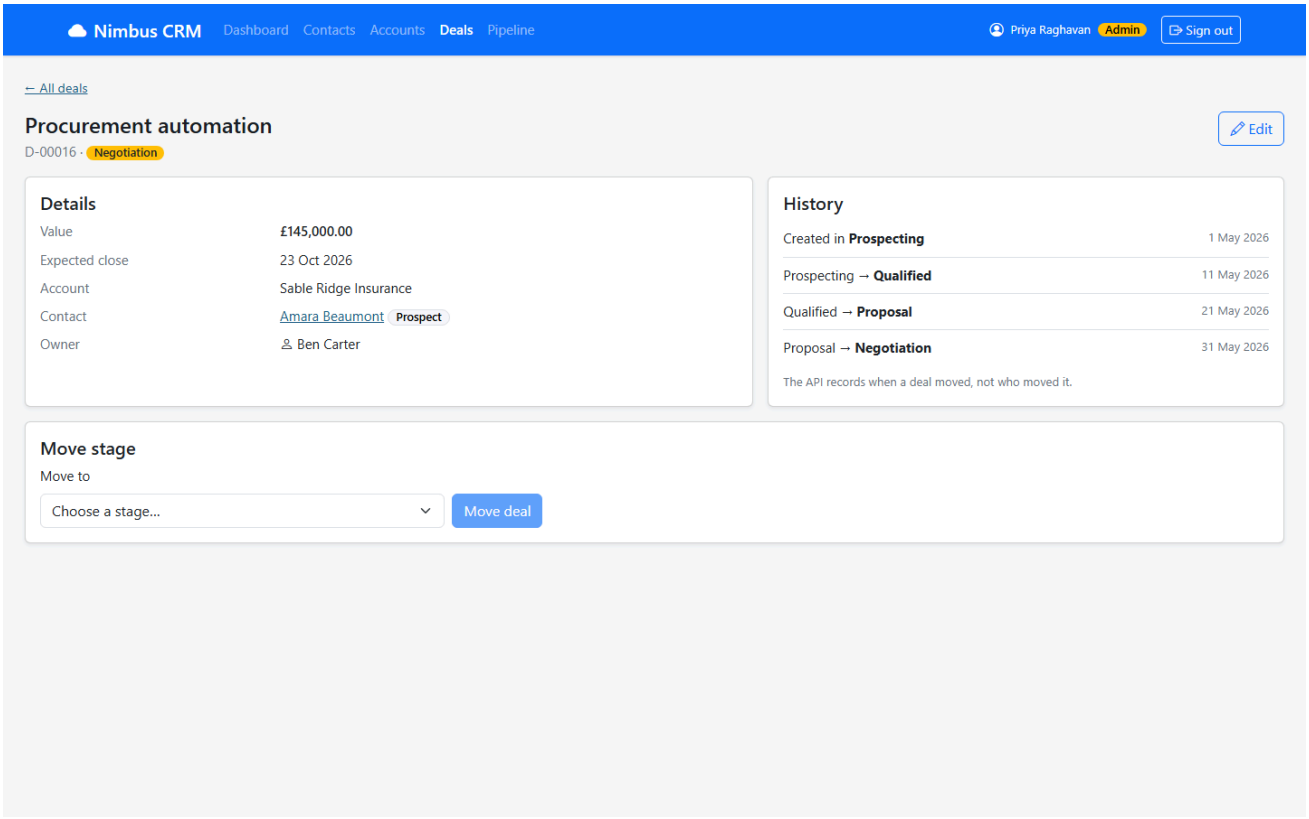
Scenario: Winning and losing
  When I win a deal, I must give the close date
  When I lose a deal, I must give a reason
  And a closed deal (Won or Lost) cannot be moved again

Scenario: Effect on the customer
  When I win a deal for a prospect
  Then that contact is shown as a customer everywhere in the app

Scenario: Refused by the server
  When the server refuses a move
  Then I see its explanation and the deal is unchanged
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`POST /api/deals/{ref}/stage` with `{stage, closeDate?, lostReason?}`. Refusals → 409 `DEAL_STAGE_TRANSITION_INVALID` or 400 with fields. Rules: KT handover, section 8.

Notes

The brief does not list the stages or the rules. Confirm them before building (NIM-100).

NIM-123 · Pipeline board and the Monday number

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	Highest
Story points	5
Depends on	NIM-120

User story

> As the Head of Inside Sales, I want to see what the pipeline is worth, broken down by stage, so that I have my number for the Monday call without working it out by hand.

Context

Priya: "I need to see the total value of the pipeline at any moment, broken down by stage. This is the number I take into the Monday leadership call… If I only get one thing from this project, this is it. Ideally the board updates in real time."

Acceptance criteria

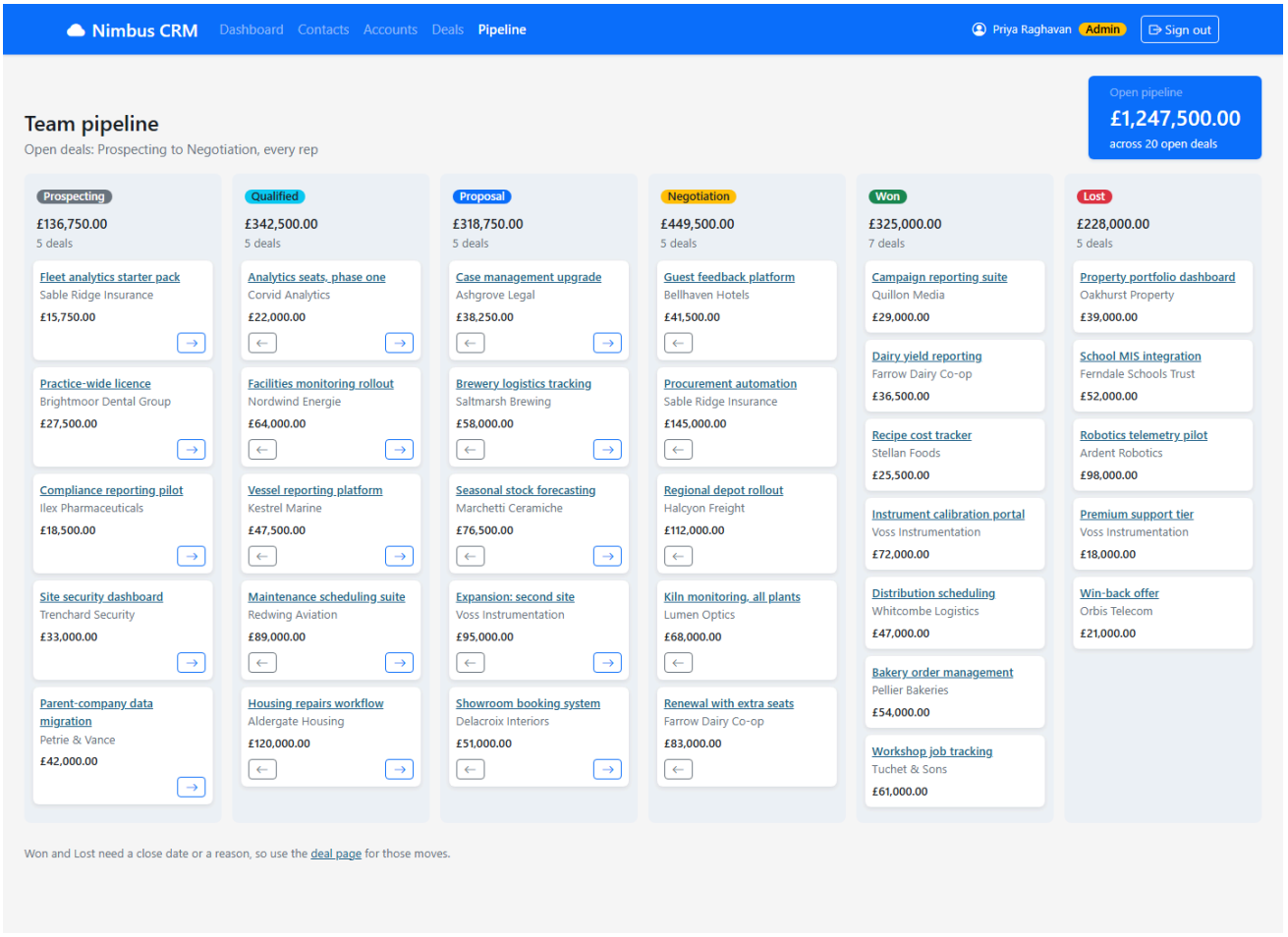
```
Scenario: The board
  When I open the pipeline
  Then I see every stage in pipeline order with the number of deals and their total value
  And I see the pipeline total Priya asked for

Scenario: Numbers match the system of record
  Then every figure equals what the API reports, to the penny
  And on a freshly seeded database, Priya's figures agree with the KT handover's test data

Scenario: Each person's board
  Given I am signed in as a rep
  Then the board shows my pipeline only
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/deals/summary` (count and value per stage plus open totals, scoped to the caller), `GET /api/deals?stage=`.

Notes

"The total value of the pipeline" is not defined precisely in the brief. Agree the definition with Priya and record it in the spec.

NIM-124 · Deals and activity on the contact page

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	High
Story points	3
Depends on	NIM-112, NIM-120

User story

> As a rep, I want to see a contact's deals and log a call or email on their page, so that I see the whole relationship and keep the history complete without leaving it.

Context

Priya: "There is usually one person there who is actually driving it, so I should be able to see the deal from their record too."

Acceptance criteria

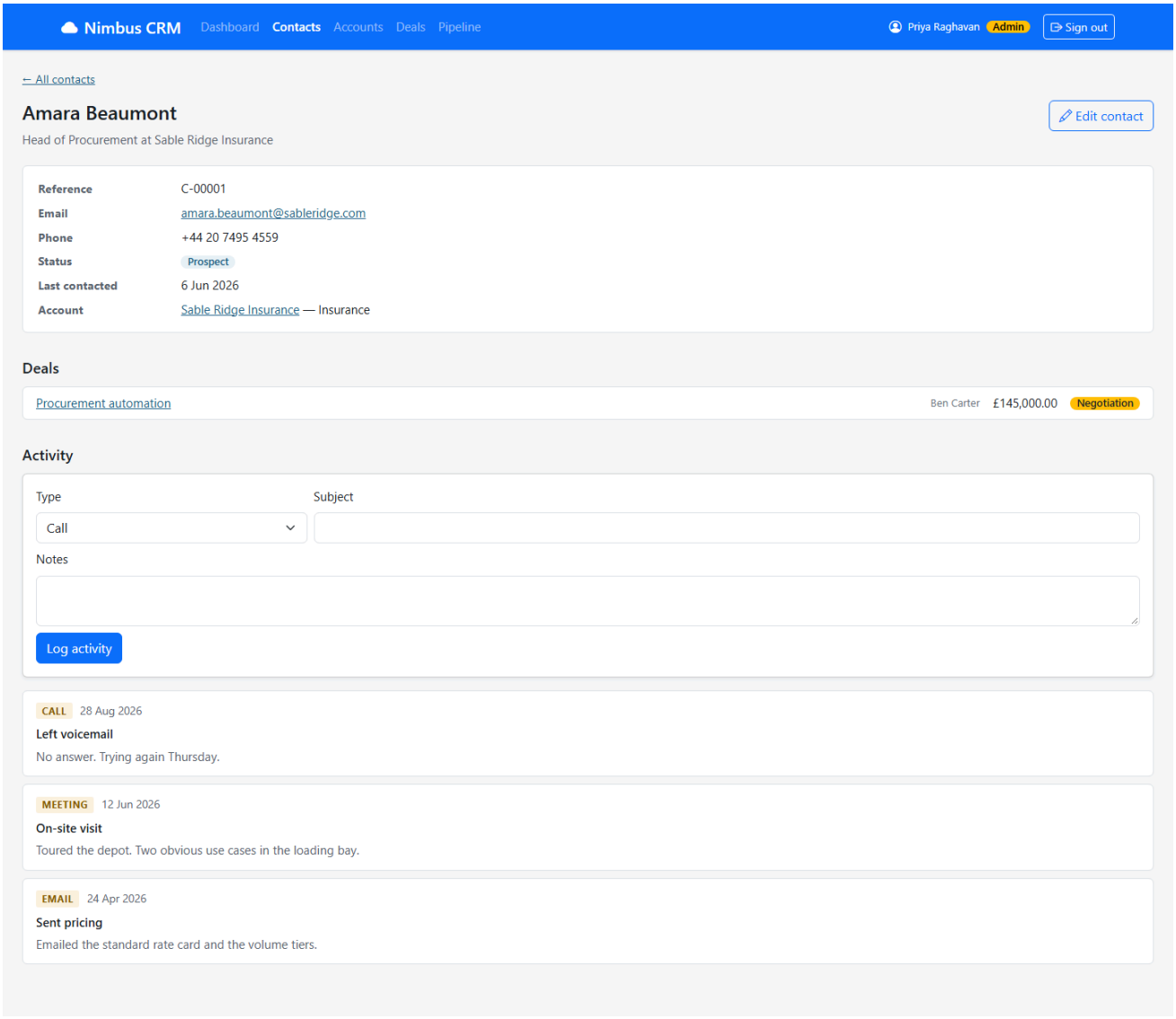
```
Scenario: Deals on the contact
  When I open a contact
  Then I see the deals they are driving that I am allowed to see, with stage and value, each leading to the de

Scenario: Log an activity
  When I log a call with a subject and notes
  Then it appears at the top of their history straight away
  And the contact's "last contacted" date reflects it

Scenario: Invalid activity
  When I try to log an activity without a subject
  Then I am told what is missing
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/contacts/{ref}/deals`, `POST /api/contacts/{ref}/activities` with `{type, subject, notes?, occurredAt?}`.

NIM-125 · Home page

Type	Story
Epic	Deal pipeline
Sprint	Sprint 2
Priority	Medium
Story points	3
Depends on	NIM-123

User story

> As anyone in Inside Sales, I want a home page that tells me how we are doing when I sign in, so that I know where to focus today.

Context

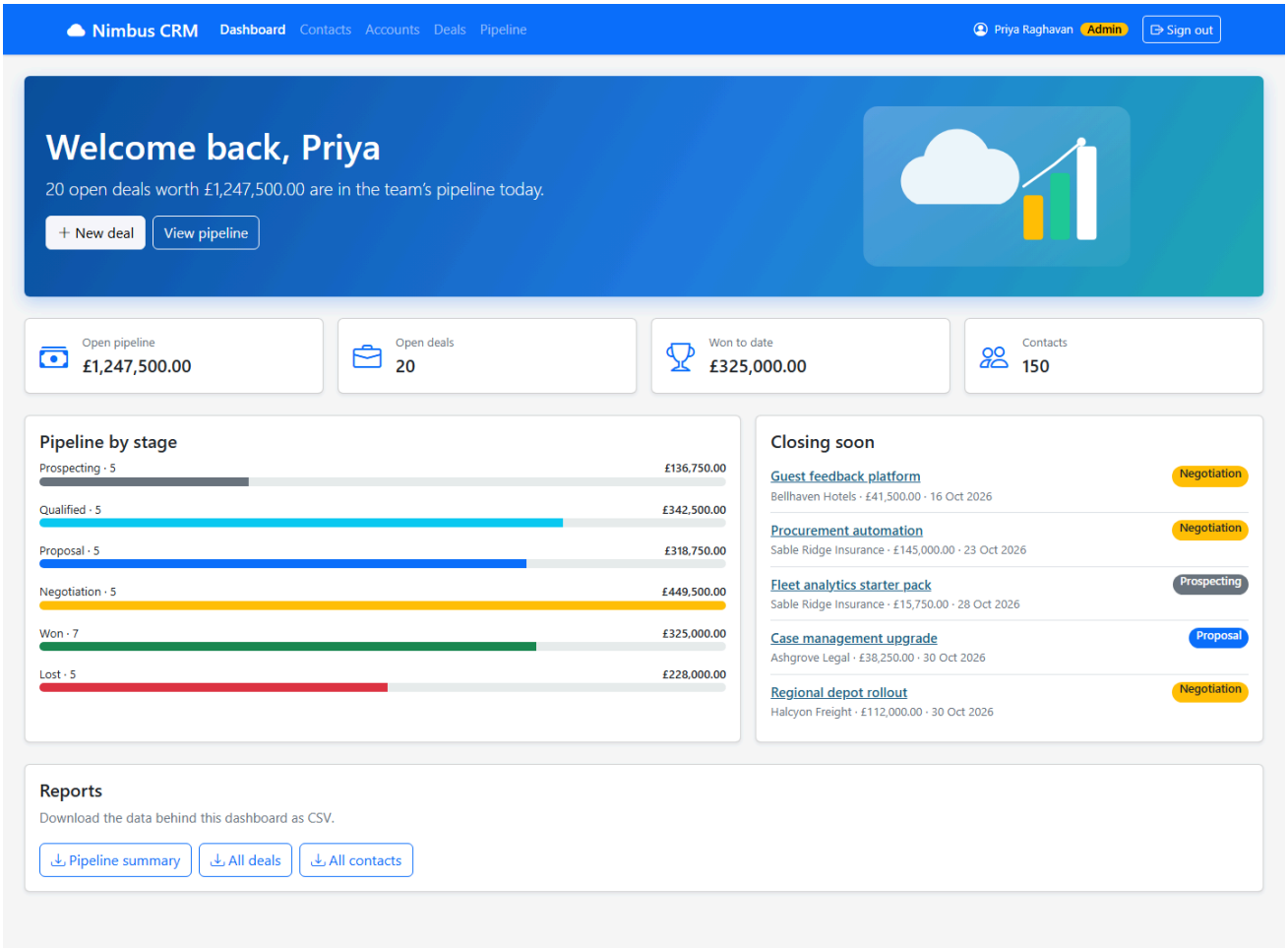
Raised by Priya in passing: "Can the first page just tell me how we're doing?" No metrics were specified.

Acceptance criteria

```
Scenario: Agreed content
  Given the team has agreed with Priya which figures and lists the home page shows
  When I sign in
  Then I see those figures, scoped to what my role allows
  And every figure matches the screens it summarises
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Notes

The design shows one possibility. Content must be agreed with the product owner before it is built.

NIM-130 · Create, edit and delete deals

Type	Story
Epic	Data management (admin)
Sprint	Sprint 3
Priority	Highest
Story points	5
Depends on	NIM-121

User story

> As Priya, I want to add deals, correct them and remove ones created by mistake, so that the CRM stays the single source of truth.

Context

Priya: "Every deal has a value and a date we think it will close. Both of those change constantly, which is fine, I just need them to be editable."

Acceptance criteria

```
Scenario: Create
  When I create a deal
  Then I choose the company first, and then only people who work there
  And I give a title, a value in pounds and an expected close date
  And the new deal starts in Prospecting

Scenario: Validation
  When I give a zero value, a date in the past or no title
  Then each problem is shown next to the field it concerns, and nothing is saved

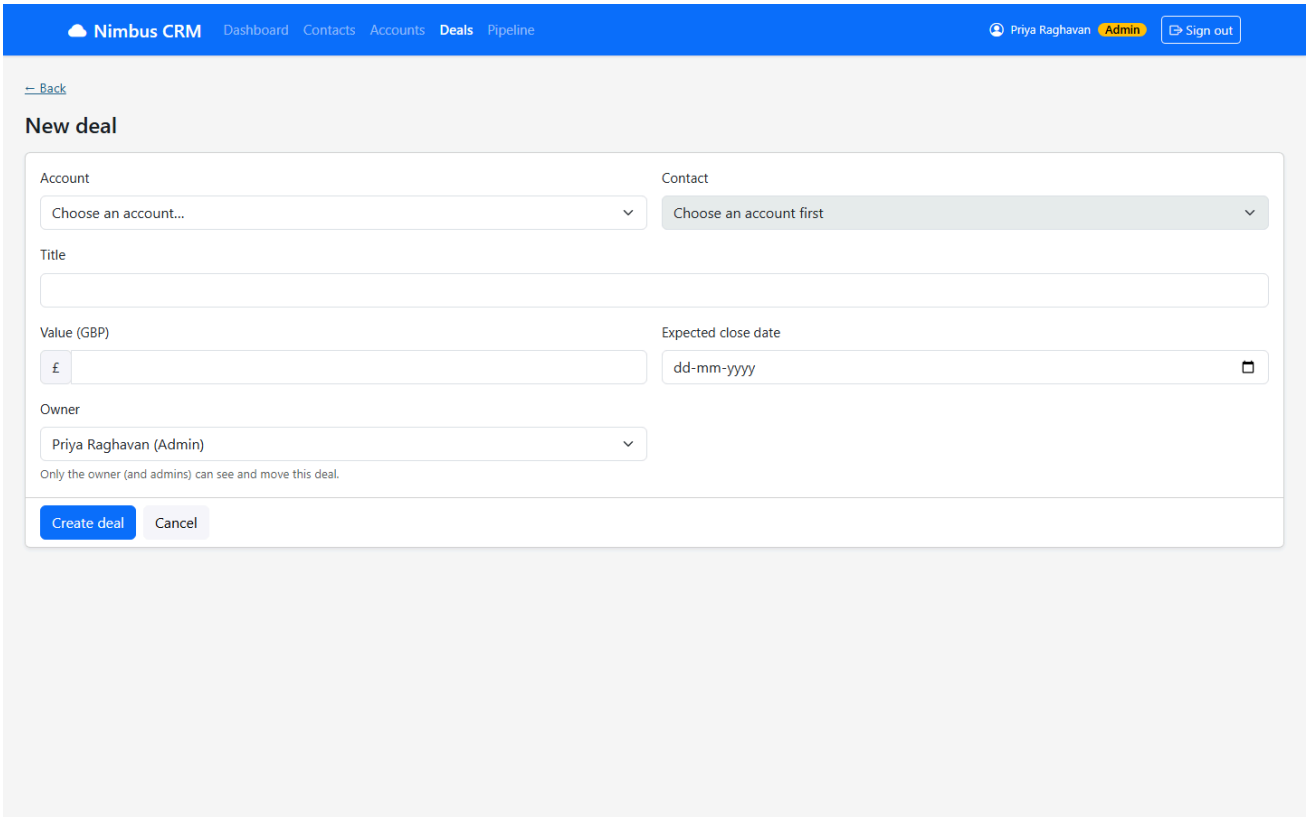
Scenario: Edit
  When I change a deal's title, value or expected close date
  Then the change is saved
  And a Won or Lost deal cannot be edited

Scenario: Delete
  When I delete a deal that is still in Prospecting, after confirming
  Then it and its history are gone
  And deals past Prospecting cannot be deleted

Scenario: No double submissions
  When I submit twice quickly
  Then only one deal is created
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`POST /api/deals`, `PUT /api/deals/{ref}`, `DELETE /api/deals/{ref}`, `GET /api/accounts/{ref}/contacts`. ADMIN only. 400 with fields, 409 `DEAL_CLOSED` / `DEAL_DELETE_NOT_ALLOWED`.

NIM-131 · Assign a deal owner

Type	Story
Epic	Data management (admin)
Sprint	Sprint 3
Priority	High
Story points	2
Depends on	NIM-130

User story

> As Priya, I want to choose which rep owns a deal and change it later, so that work is shared out and every deal has someone responsible.

Context

Ownership decides who can see and move a deal.

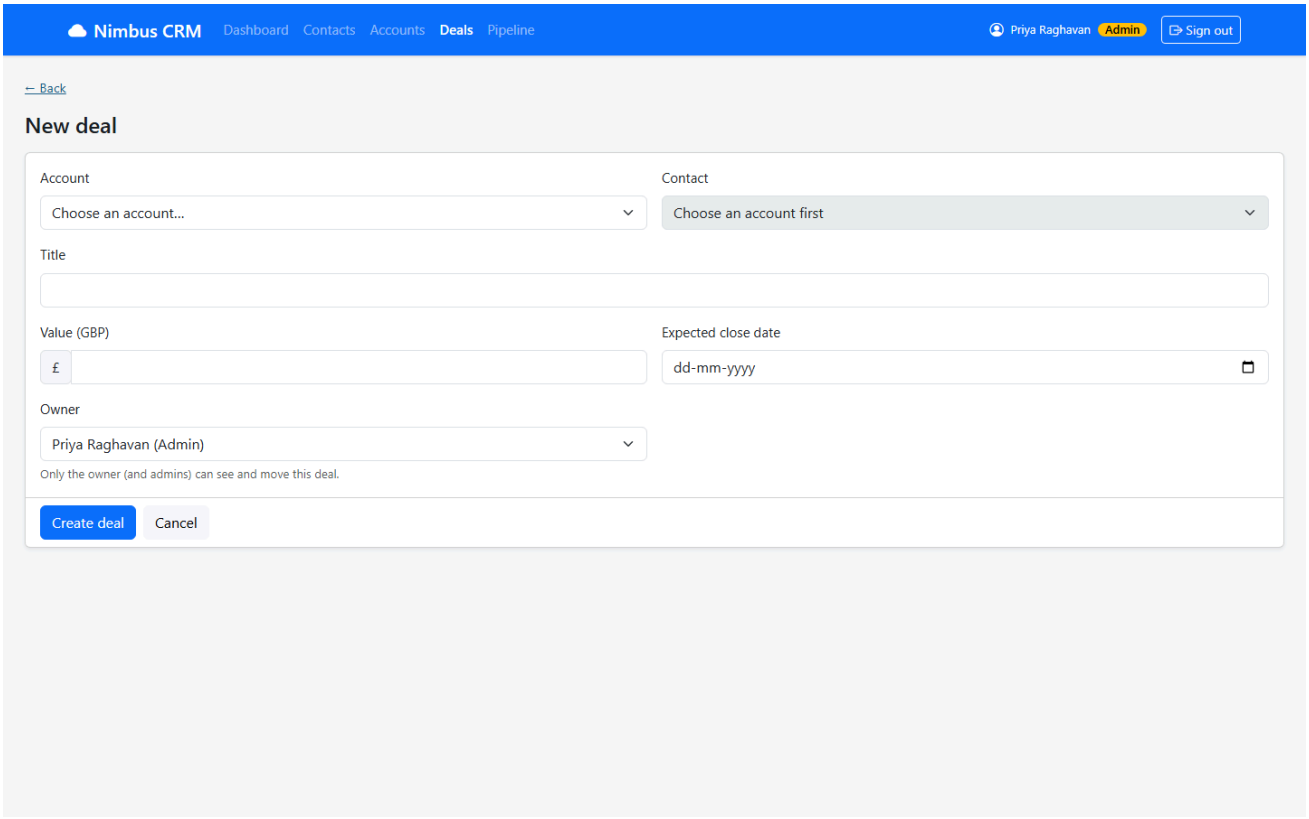
Acceptance criteria

```
Scenario: Choose owner
  When I create or edit a deal
  Then I can choose its owner from the people who can sign in

Scenario: Effect of ownership
  Given I create a deal owned by Ben
  When Ben signs in
  Then he can see and move it
  And Aisha cannot see it
```

Design

Design intent (not a pixel spec; states not shown are yours to design):



Backend notes

`GET /api/users` (ADMIN). `ownerRef` on create and update. Unknown owner → 400 `fields.ownerRef`.

NIM-132 · Add and edit accounts

Type	Story
Epic	Data management (admin)
Sprint	Sprint 3
Priority	Medium
Story points	3
Depends on	NIM-113

User story

> As Priya, I want to add a new company and keep company details up to date, so that new customers can be set up without asking IT.

Context

Accounts cannot be deleted.

Acceptance criteria

```
Scenario: Add
  When I add an account with a name
  Then it appears in the directory with a new reference

Scenario: Edit
  When I change an account's details
  Then the change is saved

Scenario: Validation
  When the name is missing or a value is too long
  Then I am told which field and why
```

Backend notes

`POST /api/accounts`, `PUT /api/accounts/{ref}`. ADMIN only.

NIM-133 · Add and edit contacts

Type	Story
Epic	Data management (admin)
Sprint	Sprint 3
Priority	Medium
Story points	3
Depends on	NIM-112, NIM-132

User story

> As Priya, I want to add people to a company and correct their details, so that the record is right.

Context

New contacts start as prospects.

Acceptance criteria

```
Scenario: Add
  When I add a contact to an account
  Then they appear on that account and in the contact list as a prospect

Scenario: Edit
  When I correct a contact's details or change their status
  Then the change is saved

Scenario: Validation
  When the email address is not valid or a required field is missing
  Then I am told which field and why
```

Backend notes

`POST /api/contacts`, `PUT /api/contacts/{ref}`. ADMIN only.

NIM-134 · Import leads from the agency

Type	Story
Epic	Data management (admin)
Sprint	Sprint 3
Priority	Low
Story points	5
Depends on	NIM-133

User story

> As Priya, I want to upload the marketing agency's monthly file of leads, so that nobody has to type them in.

Context

The agency sends a CSV every month. An example is in `docs/sample-leads.csv`. Stretch goal.

Acceptance criteria

```
Scenario: Upload
  When I upload the agency's file
  Then I am told how many contacts were created, skipped and rejected, and why each problem row was rejected

Scenario: Unreadable file
  When I upload a file that is not in the expected format
  Then I am told what is wrong with it
```

Backend notes

`POST /api/contacts/import` with `Content-Type: text/csv`; returns a plain-text report. 400 `IMPORT_FORMAT_INVALID`.

Notes

How partial failures should be presented is not specified. Ask.

NIM-140 · Automated tests, then audit them

Type	Task
Epic	Quality & release
Sprint	Sprint 3
Priority	High
Story points	5
Depends on	—

Context

Every ticket has acceptance criteria. This ticket makes sure they are proven by tests that would fail if the behaviour broke.

Acceptance criteria

```
Scenario: Traceability
  Then every acceptance criterion in this backlog maps to at least one automated test, recorded in the spec's

Scenario: End to end
  Then at least sign-in, role differences and one deal moving stage are tested end to end against the real API

Scenario: Audit
  Then each test has been checked to fail when the behaviour it covers is broken
  And any test that passes for the wrong reason is fixed and noted
```

NIM-141 · Review and harden

Type	Task
Epic	Quality & release
Sprint	Sprint 3
Priority	High
Story points	3
Depends on	NIM-140

Context

Before release, the code and its use of the API are reviewed the way a security-minded senior developer would.

Acceptance criteria

```
Scenario: Code review
  Then the code has been reviewed against docs/review-checklist.md and findings are recorded with file, line a

Scenario: Data exposure
  Then every API response the app receives has been checked for data the signed-in user should not have
  And any backend issue found has been raised as a bug report with evidence

Scenario: Failure behaviour
  Then the app's behaviour has been checked for expired sessions, 403, 404, 409, 500 and the API being down
```

NIM-142 · Release

Type	Task
Epic	Quality & release
Sprint	Sprint 3
Priority	High
Story points	2
Depends on	NIM-141

Context

Hand the app to Inside Sales.

Acceptance criteria

```
Scenario: Ready to hand over
  Then the README explains how to run, test and build the app
  And the spec, prompt journal, bug reports and test evidence are in the repository
  And a five-minute demo shows a rep's day and Priya's Monday number
```